Python Graph Library

Comprehensive API reference for the directed Graph implementation, including full exception mapping.

Class: `Graph`

A directed graph implementation utilizing nested adjacency dictionaries to track isolated vertices, incoming edges, outgoing edges, and dedicated edge weights. Includes fully robust error handling against structural violations.

Properties

`vertex_count` Property

Returns the total number of unique vertices in the graph.

`edge_count` Property

Returns the total number of directed edges currently established in the graph.

Vertex Operations

`add_vertex(vertex_id: int )`

Registers a new isolated vertex in the graph topology.

RAISES
- `ValueError` `"ERROR: Vertex already exist"` - If the vertex_id is already present.

`remove_vertex(vertex_id: int )`

Deletes a vertex and cleans up all interconnected incoming and outgoing edges across the graph.

RAISES
- `ValueError` `"ERROR: Vertex doesn't exist"` - If attempting to remove a vertex_id that isn't registered.

`get_id_degree(vertex_id: int ) → int`

Retrieves the total number of incoming edges pointing towards the specified vertex.

RAISES
- `ValueError` `"ERROR: Invalid vertex id!"` - If the referenced vertex_id does not exist.

`get_out_degree(vertex_id: int ) → int`

Retrieves the total number of outgoing edges originating from the specified vertex.

RAISES
- `ValueError` `"ERROR: Invalid vertex id!"` - If the referenced vertex_id does not exist.

Edge Operations

`add_edge(source_vertex_id: int , target_vertex_id: int , cost: int )`

Establishes a directed edge between two existing vertices and assigns it a numerical weight.

RAISES
- `ValueError` `"ERROR: Invalid source vertex id!"` - If the source_vertex_id does not exist.
- `ValueError` `"ERROR: Invalid target vertex id!"` - If the target_vertex_id does not exist.
- `ValueError` `"ERROR: Edge already exist"` - If an edge already exists linking the exact source and target.

`remove_edge(source_vertex_id, target_vertex_id)`

Permanently deletes a specific directed edge between a source and target vertex.

RAISES
- `ValueError` `"ERROR: Invalid source vertex id!"` - If the source_vertex_id does not exist.
- `ValueError` `"ERROR: Invalid target vertex id!"` - If the target_vertex_id does not exist.
- `ValueError` `"ERROR: Edge doesn't exist"` - If no edge connects the specified vertices.

`get_edge_cost(source_vertex_id: int , target_vertex_id: int ) → int`

Returns the weight/cost value currently assigned to the specified edge.

RAISES
- `ValueError` `"ERROR: Invalid source vertex id!"`
- `ValueError` `"ERROR: Invalid target vertex id!"`
- `ValueError` `"ERROR: Edge doesn't exist"`

`set_edge_cost(source_vertex_id, target_vertex_id, new_cost: int )`

Overwrites the weight of an existing directed edge.

RAISES
- `ValueError` `"ERROR: Invalid source vertex id!"`
- `ValueError` `"ERROR: Invalid target vertex id!"`
- `ValueError` `"ERROR: Edge doesn't exist"`

Iterators

`vertex_iterator()`

Returns a generator yielding all vertex IDs currently in the graph.

`edge_info_iterator()`

Yields tuples structured as `(source_vertex_id, target_vertex_id, cost)` representing every edge.

`ingoing_edge_iterator(vertex_id: int )`

Yields all source vertices that point towards the specified target vertex.

RAISES
- `ValueError` `"ERROR: Invalid vertex id!"` - If the requested target vertex does not exist.

`outgoing_edge_iterator(vertex_id: int )`

Yields all target vertices that the specified source vertex points to.

RAISES
- `ValueError` `"ERROR: Invalid vertex id!"` - If the requested source vertex does not exist.

Algorithms

`bfs(source_vertex_id, target_vertex_id) → list | None`

Breadth-First Search. Discovers the shortest path between a source and a target. *Note: Internally relies on ingoing edges to trace the path backwards.*

RAISES
- `ValueError` `"ERROR: Invalid source vertex id!"`
- `ValueError` `"ERROR: Invalid target vertex id!"`

RETURNS
- `list` containing the sequence of vertices forming the path.
- `None` if no connecting path exists.

File I/O & Utils

`save_graph(file_name: str )`

Writes the graph to disk in a standard text format. Header: `[vertices] [edges]` , Body: `[source] [target] [cost]` .

`load_graph(file_name: str )`

Wipes the current graph and rebuilds it based on the parsed data from the specified text file.

RAISES
- `FileNotFoundError` `"file {file_name} is not found"` - If the file does not exist on disk.
- `ValueError` `"ERROR: Couldn't load right number of vertexes"` - If the parsed header mismatch the resulting graph size.
- `ValueError` `"ERROR: Couldn't load right number of edges"` - If parsed header edges mismatch the loaded edges.

`copy() → Graph`

Instantiates and returns a completely disconnected deep-copy replica of the current graph.

`clear_graph()`

Erases all internal dictionaries, instantly dropping all vertices and edges.

Main CLI Module (`main.py`)

Provides the interactive CLI application wrapper for user engagement.

`generate_random_graph(graph: Graph , n: int , m: int )`

Erases the graph and rapidly generates a random topology. Edge weights are randomized between 1 and 100. Validates that `m <= n * n` .

`print_menu()`

Outputs the 12-point interactive terminal interface.